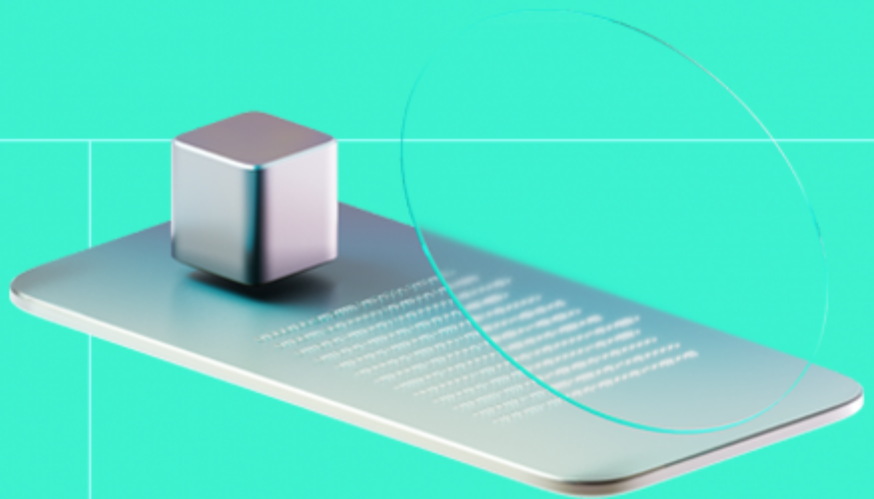




Smart Contract Code Review And Security Analysis Report

Customer: Tokenize.it

Date: 24/12/2025



We express our gratitude to the Tokenize.it team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

tokenize.it is a tokenized cap-table protocol where a compliance-gated ERC20 token (via AllowList) can be minted with configurable platform fees (via FeeSettings) and traded peer-to-peer on a secondary market through a reusable TokenSwap contract.

Document

Name	Smart Contract Code Review and Security Analysis Report for Tokenize.it
Audited By	Ataberk Yavuzer
Approved By	Ivan Bondar
Website	https://tokenize.it/en
Changelog	18/12/2025 - Preliminary Report 24/12/2025 - Preliminary Report
Platform	EVM
Language	Solidity
Tags	ERC-20, ERC-2771, Upgradable, Meta Transactions, Swaps
Methodology	https://docs.hacken.io/methodologies/smart-contracts

Review Scope

Repository	https://github.com/corpus-io/tokenize.it-smart-contracts
Initial Commit	177652a
Remediation Commit	11fbe80

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

8	4	4	0
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	0
Medium	0
Low	2

Vulnerability	Severity Status	
F-2025-14346 - TokenSwap configuration can be changed during token sale	Low	Fixed
F-2025-14355 - MINTALLOWER_ROLE has unlimited minting power with no on-chain restrictions	Low	Accepted
F-2025-14347 - Missing event emitting	Info	Fixed
F-2025-14349 - TokenSwap `buy()` / `sell()` allow zero token amount	Info	Accepted
F-2025-14350 - TokenSwap `sell()` can execute with zero currency payout due to rounding-down	Info	Accepted
F-2025-14351 - Custom errors in Solidity for gas efficiency	Info	Fixed
F-2025-14352 - AllowList batch set/remove emits per-item `Set` events	Info	Fixed
F-2025-14354 - TokenSwap uses unsafe `transferFrom()` for token transfers instead of `safeTransferFrom()`	Info	Accepted

Documentation quality

- Functional requirements are well explained.
- The technical description is well provided.
- System invariants are described.

Code quality

- The contracts heavily rely on OZ upgradeable modules, and follow a consistent “clone/initializer + _disableInitializers() logic contract” template across files.
- Most critical state changes emit events, but several changes aren’t fully “audit-friendly” (no old-value fields; manager add/remove has no events).

Test coverage

Code coverage of the project is **96.56%**.

- Deployment and basic user interactions are covered with tests.
- Randomized fuzzing tests are covered with tests.
- System invariants were tested.

Table of Contents

System Overview	6
Privileged Roles	6
Potential Risks	8
Findings	9
Vulnerability Details	9
Disclaimers	26
Appendix 1. Definitions	27
Severities	27
Potential Risks	27
Appendix 2. Scope	28
Appendix 3. Additional Valuables	29

System Overview

tokenize.it is a tokenized cap-table trading module with the following contracts:

- **AllowList** — owner-managed registry mapping addresses to a 256-bit attribute bitmask (incl. a reserved TRUSTED_CURRENCY bit). Other contracts read this to decide whether an address/currency is permitted.
- **FeeSettings** — owner-managed fee configuration contract implementing IFeeSettingsV2 (and IFeeSettingsV1 for compatibility). Stores default fees + optional per-token discounts and per-token fee-collector overrides. Enforces max fee caps and a 12-week delay for fee increases.
- **Token** — upgradeable ERC20 (ERC20Permit, ERC20Snapshot) with pause + role-based controls and allowlist-gated transfers. Minting is restricted via mintingAllowance or MINTALLOWER_ROLE; a token fee is minted to the token fee collector per FeeSettings.
- **TokenSwap** — reusable standing order contract for peer-to-peer secondary swaps of already-issued Token against a fixed ERC20 currency at a fixed price, charging the **crowdinvesting** fee from FeeSettings. Supports both “sell order” (holder provides tokens) and “buy order” (holder provides currency).

Privileged roles

- **AllowList owner**
 - Can arbitrarily add/remove/modify any address’s attributes, including marking currencies as trusted.
 - Effectively controls who can transact (subject to how Token consumes allowList).
- **FeeSettings owner**
 - Can change default fee rates and default fee collectors.
 - Can appoint/remove managers.
 - Can propose fee increases only with ≥ 12 -week delay; decreases can be immediate.
- **FeeSettings managers**
 - Can set/remove per-token fee discounts and per-token fee collectors (high leverage over fee routing).
- **Token DEFAULT_ADMIN_ROLE**
 - Can grant/revoke all roles and authorize upgrades (UUPS).
 - Can change AllowList reference.
 - Can accept a suggested FeeSettings contract.
- **Token REQUIREMENT_ROLE / PAUSER_ROLE / BURNER_ROLE / TRANSFERERADMIN_ROLE**
 - **REQUIREMENT_ROLE**: can set the requirements bitmask for gating transfers.
 - **PAUSER_ROLE**: can pause/unpause transfers/mints/burns.
 - **BURNER_ROLE**: can burn tokens from any address.
 - **TRANSFERERADMIN_ROLE**: can grant TRANSFERER_ROLE (bypasses allowlist requirements).
- **TokenSwap owner**
 - Can pause/unpause trading.

- Can change holder, receiver, and tokenPrice (per current implementation), which is a high trust/centralization risk for live orders.

Potential Risks

- **Scope Definition and Security Guarantees:** The audit does not cover all code in the repository. Contracts outside the audit scope may introduce vulnerabilities, potentially impacting the overall security due to the interconnected nature of smart contracts.
- **System Reliance on External Contracts:** The functioning of the system significantly relies on specific external contracts. Any flaws or vulnerabilities in these contracts adversely affect the audited project, potentially leading to security breaches or loss of funds. (Examples here include the immutable EIP-2771 trusted forwarder and OpenZeppelin implementations.)
- **Single Points of Failure and Control:** The project is fully or partially centralized, introducing single points of failure and control. This centralization can lead to vulnerabilities in decision-making and operational processes, making the system more susceptible to targeted attacks or manipulation. (AllowList owner, FeeSettings owner/managers, Token admin roles, TokenSwap owner.)
- **Administrative Key Control Risks:** The digital contract architecture relies on administrative keys for critical operations. Centralized control over these keys presents a significant security risk, as compromise or misuse can lead to unauthorized actions or loss of funds.
- **Absence of Time-lock Mechanisms for Critical Operations:** Without time-locks on critical operations, there is no buffer to review or revert potentially harmful actions, increasing the risk of rapid exploitation and irreversible changes. (E.g., TokenSwap parameters can be updated by owner; Token is upgradeable.)
- **Flexibility and Risk in Contract Upgrades:** The project's contracts are upgradable, allowing the administrator to update the contract logic at any time. While this provides flexibility in addressing issues and evolving the project, it also introduces risks if upgrade processes are not properly managed or secured, potentially allowing for unauthorized changes that could compromise the project's integrity and security.

Findings

Vulnerability Details

F-2025-14346 - TokenSwap configuration can be changed during token sale - Low

Description:

`TokenSwap` is intended to represent a standing buy/sell order with fixed counterparties and economics. The documentation/spec further implies that parameter changes are performed **while paused** to prevent unsafe live reconfiguration.

However, `TokenSwap` allows the **owner** to change `holder`, `receiver`, and `tokenPrice` at any time (even while unpaused). This creates a high-impact trust-boundary issue.

If an address has previously granted approvals to the `TokenSwap` contract (token allowance for `holder` in sell-orders or currency allowance for `holder` in buy-orders), a malicious or compromised owner can:

- **Redirect proceeds:** change `receiver` to an attacker's address so that currency/tokens are routed to the attacker.
- **Abuse existing approvals:** change `holder` to an address that has previously approved this `TokenSwap` for token/currency transfers, and then execute `buy()` / `sell()` under the new configuration.

TokenSwap.sol:

```
function setReceiver(address _receiver) external onlyOwner { ... }  
function setTokenPrice(uint256 _tokenPrice) external onlyOwner { ... }  
function setHolder(address _holder) external onlyOwner { ... }
```

Assets:

- TokenSwap.sol [<https://github.com/corpus-io/tokenize.it-smart-contracts/tree/main/contracts>]

Status:

Fixed

Classification

Impact Rate:

4/5

Likelihood Rate: 2/5

Exploitability: Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Require `whenPaused` on all parameter-changing functions (`setReceiver` , `setHolder` , `setTokenPrice` , and any future setter for `currency`).

Resolution: The finding is fixed in commit **4723bc7**. The holder variable was set to immutable, and the `setHolder` function was removed to mitigate this finding. Other variables and functionalities were not found risky by the client, with additional statements:

setPrice:

This has already been taken into account by extending the buy and sell functions with an additional argument (`_maxCurrencyAmount` for buy, `_minCurrencyAmount` for sell).

`sell(uint256 _tokenAmount, uint256 _minCurrencyAmount, address _currencyReceiver )` : the user states "I want to sell `_tokenAmount` tokens and I expect to receive at least `_minCurrencyAmount` currency." If the owner were to frontrun with a price that results in a payout of less than `_minCurrencyAmount`, the sell reverts. Thus, the user is protected of malicious price changes. (If the owner frontruns with a higher price, the user receives a higher payout, which is okay.)

`function buy(uint256 _tokenAmount, uint256 _maxCurrencyAmount, address _tokenReceiver)` : the user states "I want to buy `_tokenAmount` tokens and I want to pay no more than `_maxCurrencyAmount` for that. Again, if a price change would result in the user having to pay more than expected, the transaction would revert.

We see no potential for abuse here.

setReceiver:

`setReceiver` is onlyOwner. That means an attacker couldn't set a receiver to harm the owner. If the owner sets a receiver, we must assume they do it intentionally. We don't see how this could have unexpected consequences for any party, except maybe if the owner sets receiver to some address they don't control and that doesn't want to receive funds. That however is always a risk on ethereum, as anyone

can send anything to any address, with or without using a smart contract.

F-2025-14355 - MINTALLOWER_ROLE has unlimited minting power with no on-chain restrictions - Low

Description:

The `Token.mint()` function implements a two-tier minting system where addresses with `MINTALLOWER_ROLE` have **unlimited minting power** without any allowance checks, while non-privileged addresses must consume minting allowances. This creates significant centralization risks and potential for supply manipulation.

The current implementation (**Token.sol**):

```
function mint(address _to, uint256 _amount) external {
    if (!hasRole(MINTALLOWER_ROLE, _msgSender())) {
        require(mintingAllowance[_msgSender()] >= _amount, "MintingAllowance too low");
        mintingAllowance[_msgSender()] -= _amount;
    }
    // ... minting proceeds without restriction for MINTALLOWER_ROLE
    _checkIfAllowedToTransact(_to);
    _mint(_to, _amount);
    // collect fees
    uint256 fee = feeSettings.tokenFee(_amount, address(this));
    _mint(feeSettings.tokenFeeCollector(address(this)), fee);
}
```

Additionally, `MINTALLOWER_ROLE` can call `increaseMintingAllowance()` to grant themselves or others unlimited allowances:

```
function increaseMintingAllowance(address _minter, uint256 _allowance) external onlyRole(MINTALLOWER_ROLE) {
    mintingAllowance[_minter] += _allowance;
    emit MintingAllowanceChanged(_minter, mintingAllowance[_minter]);
}
```

Any compromised `MINTALLOWER_ROLE` address can mint unlimited tokens at any time.

Assets:

- Token.sol [<https://github.com/corpus-io/tokenize.it-smart-contracts/tree/main/contracts>]

Status:

Accepted

Classification

Impact Rate:	4/5
Likelihood Rate:	2/5
Exploitability:	Semi-Dependent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation: This is a business logic-related issue. Therefore, the solution might not be easy and one-sided.

The following countermeasures can make this system more error-proof:

- Remove the bypass and require **all** minters (including `MINTALLOWER_ROLE`) to consume allowances.
- If unlimited minting by `MINTALLOWER_ROLE` is intentional, add protective measures, introduce a `maxTotalSupply` to prevent minting huge numbers.

Resolution: The finding was acknowledged by the client, and its risk was explained further by the client in the following statement:

DEFAULT_ADMIN_ROLE can do that anyway. DEFAULT_ADMIN can do almost anything. So the risk is real anyway, as it is in almost all ERC20 tokens. It doesn't increase or decrease by defining a MINTALLOWER_ROLE that has a part of the power of DEFAULT_ADMIN.

In fact, if a separate MINTALLOWER would go rogue, the DEFAULT_ADMIN could still remove them and possibly burn the tokens, or possibly use an existing snapshot, or update the contract logic.

On deployment, all roles are assigned to DEFAULT_ADMIN, and currently in none of our deployments does anyone else hold the MINTALLOWER_ROLE. All contracts (Private offer, Crowdfunding, Vesting) are MINTERS, not MINTALLOWERS.

[F-2025-14347](#) - Missing event emitting - Info

Description:

`Token.increaseMintingAllowance()` and `Token.decreaseMintingAllowance()` emit `MintingAllowanceChanged`, but consuming allowance inside `Token.mint()` does not.

Additionally, `FeeSettings.addManager()` and `FeeSettings.removeManager()` update the `managers` mapping, but do not emit events.

This is not a correctness or security issue (the ERC20 `Transfer` event still shows minting), but it makes off-chain accounting harder (especially for “remaining allowance” dashboards) and can cause discrepancies between “allowance changes” and “allowance usage” telemetry.

Token.sol:

```
if (!hasRole(MINTALLOWER_ROLE, _msgSender())) {
    require(mintingAllowance[_msgSender()] >= _amount, "MintingAllowance too low");
    mintingAllowance[_msgSender()] -= _amount; // no MintingAllowanceChanged emitted here
}
```

FeeSettings.sol:

```
function addManager(address _manager) external onlyOwner {
    managers[_manager] = true;
}

function removeManager(address _manager) external onlyOwner {
    delete managers[_manager];
}
```

Assets:

- FeeSettings.sol [<https://github.com/corpus-io/tokenize.it-smart-contracts/tree/main/contracts>]
- Token.sol [<https://github.com/corpus-io/tokenize.it-smart-contracts/tree/main/contracts>]

Status:

Fixed

Classification

Impact Rate:

1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Implement and emit the missing events for specified functions.

Resolution: The finding was resolved in the commit hash **2d41b5e**. Missing events were implemented.

F-2025-14349 - TokenSwap `buy()` / `sell()` allow zero token amount - Info

Description:

`TokenSwap` is intended to facilitate "real" secondary market fills. However, both `buy()` and `sell()` accept `_tokenAmount == 0`. These calls are effectively no-ops from a financial perspective, but they still:

- consume gas for the caller
- emit trade events (`TokensBought` / `TokensSold`) with `tokenAmount == 0`
- can pollute off-chain analytics/monitoring (volume trackers, alerting rules, indexers that assume non-zero fills)

TokenSwap.sol:

```
function buy(
    uint256 _tokenAmount,
    uint256 _maxCurrencyAmount,
    address _tokenReceiver
) public whenNotPaused nonReentrant {
    // rounding up to the next whole number. Investor is charged up to one currency bit more in case of a fractional currency bit.
    uint256 currencyAmount = Math.ceilDiv(_tokenAmount * tokenPrice, 10 ** token.decimals());
```

```
function sell(
    uint256 _tokenAmount,
    uint256 _minCurrencyAmount,
    address _currencyReceiver
) public whenNotPaused nonReentrant {
    // rounding down. Seller receives at most the exact price, protecting the holder.
    uint256 currencyAmount = (_tokenAmount * tokenPrice) / (10 ** token.decimals());
```

Assets:

- TokenSwap.sol [<https://github.com/corpus-io/tokenize.it-smart-contracts/tree/main/contracts>]

Status:

Accepted

Classification

Impact Rate:

1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Add an explicit check at the start of both functions, e.g.:

- `require(_tokenAmount > 0, "Token amount must be > 0");`

Resolution: The finding was acknowledged with the following statement:

The finding is correct. If anyone calls this with 0, they can do so, and have to pay the fees.

Adding the recommended require would increase the gas cost for all legitimate users, only to protect the hypothetical user voluntarily buying 0 tokens from no significant loss. We do not like that tradeoff and will not change the code.

[F-2025-14350](#) - TokenSwap `sell()` can execute with zero currency payout due to rounding-down - Info

Description: `TokenSwap.sell()` computes the currency payout using integer division with **rounding down**:

```
currencyAmount = (_tokenAmount * tokenPrice) / (10 ** token.decimals())
```

For sufficiently small `_tokenAmount`, `currencyAmount` becomes **0**. If the caller sets `_minCurrencyAmount = 0`, the function succeeds and:

- transfers **0 currency** from `holder` to `_currencyReceiver`
- still transfers `_tokenAmount` tokens from the caller to `receiver`

The issue occurs because:

1. The function calculates: `currencyAmount = (_tokenAmount * tokenPrice) / (10 ** token.decimals())`
2. When `_tokenAmount * tokenPrice < 10 ** token.decimals()`, the division rounds down to 0
3. When `currencyAmount == 0`, the computed fee is also 0 (linear fee), so the payout check passes if `_minCurrencyAmount == 0`
4. The seller transfers tokens but receives **0** currency

TokenSwap.sol:

```
function sell(
    uint256 _tokenAmount,
    uint256 _minCurrencyAmount,
    address _currencyReceiver
) public whenNotPaused nonReentrant {
    // rounding down. Seller receives at most the exact price, protecting the
    // holder.
    uint256 currencyAmount = (_tokenAmount * tokenPrice) / (10 ** token.decimals()); // Result can be 0

    (uint256 fee, address feeCollector) = _getFeeAndFeeReceiver(currencyAmount);

    if (fee != 0) {
        currency.safeTransferFrom(holder, feeCollector, fee);
    }

    // If _minCurrencyAmount = 0, this passes even when currencyAmount = 0
    require(currencyAmount - fee >= _minCurrencyAmount, "Payout too low");

    // Transfers 0 currency to seller
```

```

    currency.safeTransferFrom(holder, _currencyReceiver, currencyAmount - fee
    );

    // But still transfers tokens from seller
    token.transferFrom(_msgSender(), receiver, _tokenAmount);

    emit TokensSold(_msgSender(), _tokenAmount, currencyAmount);
}

```

Assets:

- TokenSwap.sol [<https://github.com/corpus-io/tokenize.it-smart-contracts/tree/main/contracts>]

Status:

Accepted

Classification

Impact Rate:	2/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

Add validation to prevent zero-payout sales.

```

require(_tokenAmount > 0, "Cannot sell zero tokens");
require(currencyAmount > 0, "Payout too small");

```

Resolution:

The finding was acknowledged by the client with the following statement:

That is correct. We round down on sell to protect the holder, and up on buy for the same reason. The user (who calls sell or buy) can protect themselves by setting the min/maxCurrency. If they don't, they are exposed to a tiny risk, as this rounding only ever is a tiny dust amount. We accept that in return for lower gas fees for legitimate users.

[F-2025-14351](#) - Custom errors in Solidity for gas efficiency - Info

Description:

Starting from Solidity version **0.8.4**, the language introduced a feature known as "**custom errors**". These custom errors provide a way for developers to define more descriptive and semantically meaningful error conditions without relying on string messages. Prior to this version, developers often used the `require` statement with string error messages to handle specific conditions or validations. However, every unique string used as a revert reason consumes gas, making transactions more expensive.

Custom errors, on the other hand, are identified by their name and the types of their parameters only, and they do not have the overhead of string storage. This means that, when using custom errors instead of `require` statements with string messages, the gas consumption can be significantly reduced, leading to more gas-efficient contracts.

It was observed that long messages were used in `require()` statements that are not a gas-efficient approach.

Assets:

- FeeSettings.sol [<https://github.com/corpus-io/tokenize.it-smart-contracts/tree/main/contracts>]
- Token.sol [<https://github.com/corpus-io/tokenize.it-smart-contracts/tree/main/contracts>]
- TokenSwap.sol [<https://github.com/corpus-io/tokenize.it-smart-contracts/tree/main/contracts>]
- AllowList.sol [<https://github.com/corpus-io/tokenize.it-smart-contracts/tree/main/contracts>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

It is recommended to use **custom errors** instead of **revert strings** to reduce gas costs, especially during contract deployment. Custom errors can be defined using the error keyword and can include dynamic information.

E.g:

```
error ZeroAmount();
```

Resolution:

The finding was acknowledged by the client with the following statement:

thank you for pointing that out. We might implement it in future.

[F-2025-14352](#) - AllowList batch set/remove emits per-item `Set` events - Info

Description:

`AllowList` supports batch operations:

- `set(address[] calldata, uint256[] calldata)`
- `remove(address[] calldata)`

Internally, these loop and call the single-item `_set()` / `remove()` which emits the `Set(address indexed _addr, uint256 _attributes)` event **once per element**.

This is correct, but it has a legitimate upside in that indexers can efficiently filter logs by `_addr` because it is `indexed` (stored as a topic).

However, emitting **N separate events** also:

- increases gas cost for batch updates (N `LOG` operations instead of 1)
- produces noisier logs (especially for large migrations / periodic refreshes)
- can complicate “batch semantics” off-chain (one user action → many events)

So moving `emit` outside the loop typically reduces gas for batch sizes `N > 1`, because you pay the fixed `LOG` overhead once instead of N times. The total bytes logged are of similar order (each element still needs to be represented somewhere), but the per-event overhead is avoided.

AllowList.sol:

```
function _set(address[] calldata _addr, uint256[] calldata _attributes) internal {
    require(_addr.length == _attributes.length, "lengths do not match");
    for (uint256 i = 0; i < _addr.length; i++) {
        _set(_addr[i], _attributes[i]); // emits Set each iteration
    }
}

function remove(address[] calldata _addr) external onlyOwner {
    for (uint256 i = 0; i < _addr.length; i++) {
        remove(_addr[i]); // emits Set each iteration
    }
}
```

Assets:

- AllowList.sol [<https://github.com/corpus-io/tokenize.it-smart-contracts/tree/main/contracts>]

Status:**Fixed****Classification****Impact Rate:** 1/5**Likelihood Rate:** 1/5**Exploitability:** Independent**Complexity:** Simple**Severity:** Info**Recommendations****Remediation:** Optional optimization (tradeoff-aware):

- **Keep** the current per-address `Set(address indexed, uint256)` event for single updates and for cases where per-address filtering is important.
- **Add** an additional batch event for batch operations, and emit it once per batch, e.g.:

```
event BatchSet(address[] addrs, uint256[] attributes);  
event BatchRemove(address[] addrs);
```

Resolution:

The finding is fixed in commit hash **1973d17**. Batch events were implemented to optimise gas consumption.

[F-2025-14354](#) - TokenSwap uses unsafe `transferFrom()` for token transfers instead of `safeTransferFrom()` - Info

Description:

`TokenSwap.buy()` and `TokenSwap.sell()` use standard `transferFrom()` for token transfers, while using `SafeERC20.safeTransferFrom()` for currency transfers. This creates an inconsistency and potential safety gap.

TokenSwap.sol:

```
contract TokenSwap is ... {
    using SafeERC20 for IERC20;

    Token public token;
    IERC20 public currency;

    function buy(...) public whenNotPaused nonReentrant {
        // ...
        currency.safeTransferFrom(_msgSender(), feeCollector, fee);
        currency.safeTransferFrom(_msgSender(), receiver, currencyAmount - fee);
    }

    token.transferFrom(holder, _tokenReceiver, _tokenAmount);
}

function sell(...) public whenNotPaused nonReentrant {
    // ...
    currency.safeTransferFrom(holder, feeCollector, fee);
    currency.safeTransferFrom(holder, _currencyReceiver, currencyAmount - fee);
    token.transferFrom(_msgSender(), receiver, _tokenAmount);
}
```

While the `Token` type is a known contract that properly reverts on failure (not an arbitrary ERC20), using raw `transferFrom()`:

- does not validate return values
- is inconsistent with the `SafeERC20` pattern used elsewhere in the same functions
- could cause silent failures if the token implementation is ever upgraded to a non-reverting variant

Assets:

- TokenSwap.sol [<https://github.com/corpus-io/tokenize.it-smart-contracts/tree/main/contracts>]

Status: Accepted

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Use `SafeERC20.safeTransferFrom()` for token transfers to maintain consistency and defensive programming.

Resolution: The finding was acknowledged.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/corpus-io/tokenize.it-smart-contracts
Initial Commit	177652a3fa26f1251d573a4cd10c199bf8598ae5
Remediation Commit	11fbe80afe0b57e3d9fa411986fb224a1f0df865
Whitepaper	N/A
Requirements	README.md
Technical Requirements	deployment.md

Asset	Type
AllowList.sol [https://github.com/corpus-io/tokenize.it-smart-contracts/tree/main/contracts]	Smart Contract
FeeSettings.sol [https://github.com/corpus-io/tokenize.it-smart-contracts/tree/main/contracts]	Smart Contract
Token.sol [https://github.com/corpus-io/tokenize.it-smart-contracts/tree/main/contracts]	Smart Contract
TokenSwap.sol [https://github.com/corpus-io/tokenize.it-smart-contracts/tree/main/contracts]	Smart Contract

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the [NIST SP 800-115 - Technical Guide to Information Security Testing and Assessment](#), and the [Penetration Testing Execution Standard \(PTES\)](#). These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.